

第2章 文字列から痕跡を探そう

この章で学ぶこと

- バイナリにも文字列が残る
- URLやエラーメッセージが見える
- プログラミング言語特有の痕跡

バイナリとは

プログラムをコンパイルすると、ソースコードはバイナリ（実行ファイル）に変換されます。

RustとCでの例：

hello.rs → hello（**rustc**というコンパイラでバイナリに変換）

hello.c → hello（**gcc**というコンパイラでバイナリに変換）

バイナリは0と1の羅列で、通常のテキストエディタで開くと大半は文字化けして見えます。

しかし、一部の領域には人間が読めるASCII文字列がそのまま埋め込まれています。

▼ なんで文字列は残るのか？

ソースコードをコンパイルすると、`if` や `for` などの制御フローは機械語に変換されます。しかし文字列リテラルは変換する必要がなく、`'a'=0x61`、`'d'=0x64` のようにASCII値がそのままバイト列としてバイナリが持つ読み取り専用データ領域セクションに格納されます。だから `strings` で読み出せるわけです。

文字列を抽出してみよう

簡単なRustプログラムを用意しました。

```

1 fn main() {
2     let user = "admin";
3     let password = "Password_secret";
4
5     if user == "admin" && password == "Password_secret" {
6         println!("ログイン成功!");
7     } else {
8         println!("認証失敗: ユーザー名またはパスワードが違います");
9     }
10 }

```

`cargo build` を行くと、バイナリが `target/debug/` に生成されます。

```

~/workshop/strings > master
> ls
Cargo.lock Cargo.toml src target

~/workshop/strings > master
> cargo build
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s

```

`strings` というコマンドを用いて、この赤い `strings` という名前のバイナリを調査してみましょう！

```

~/workshop/strings/target/debug > master
> strings strings
adminPassword_secret
2>4>>
>>>, .
>>$*
>&<6>>>
0>>

```

```

fatal runtime error: Rust panics must be rethrown, aborting
fatal runtime error: Rust cannot catch foreign exceptions, aborting
thread name may not contain interior null bytes
<unknown>
Archive member size is too large
Contents/Resources/DWARF
.debug_abbrev.dwo.debug_cu_index.debug_info.dwo.debug_line.dwo.debug_loc.dwo.debug_loclists.dwo.debug_rnglists.dwo.debug
_str.dwo.debug_str_offsets.dwo.debug_tu_index
a formatting trait implementation returned an error when the underlying stream did not
failed to write whole bufferinternal error: entered unreachable code0smessageErrorCustomerrorpermission deniedconnection
refusednetwork unreachableconnection abortednot connectedaddress in useaddress not availablenetwork downbroken pipeenti

```

```
, aborting
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/alloc/src/collections/btree/navigate.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/thread/thread.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/../../backtrace/src/symbolize/mod.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/../../backtrace/src/symbolize/gimli/libc_macos.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/../../backtrace/src/symbolize/gimli/lru.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/../../backtrace/src/symbolize/gimli.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/io/mod.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/io/stdio.rs
failed printing to
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/io/io_slice/iovec.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/rt.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/thread_local/destructors/list.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/fs/unix.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/io/error/unix.rs
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/pal/unix/stack_overflow/thread_info.rs
thread '
) has overflowed its stack
/rustc/e408947bfd200af42db322daf0fadfe7e26d3bd1/library/std/src/sys/pal/unix/stack_overflow.rs
```

stringsを行うと上記のように様々な情報を見ることができます。

1枚目の画像ではコードにあったadminとPassword_secretというハードコードされたもの（定数がそのまま書かれたもの）が見えます。ただしコンパイラの最適化によって消えることがあります。

2枚目の画像ではRustが用いるエラーメッセージなどがたくさん含まれています。

3枚目の画像ではrust/e408...3bd1（コミットハッシュと呼ばれるもの）という文字列が複数あります。

この情報からRustのバージョンもわかります。

Meta

```
rustc --version --verbose :
```

```
rustc 1.94.1 (e408947bf 2026-03-25)
binary: rustc
commit-hash: e408947bfd200af42db322daf0fadfe7e26d3bd1
commit-date: 2026-03-25
host: x86_64-unknown-linux-gnu
release: 1.94.1
LLVM version: 21.1.8
```

上記以外でもstringsで確認できることはたくさんあります。ぜひ調べてみてください。

stringsの仕組み

`strings` はバイナリファイルを先頭から1バイトずつスキャンし、印字可能なASCII文字が4文字以上連続した箇所を抽出して出力するツールです。

(61=a, 64=d, 6d=m, 69=i, 6e=n) みたいな感じです。

```
00035ae0: 5000 0018 3377 7717 0000 0000 5000 0018 P...3.....P...
00035af0: 30ff ff17 cb05 0000 0000 0000 0000 0000 0.....
00035b00: 6164 6d69 6e50 4073 7377 3072 645f 7365 adminP@ssw@rd_se
00035b10: 6372 6574 e8aa 8de8 a8bc e5a4 b1e6 9597 cret.....
00035b20: 3a20 e383 a6e3 83bc e382 b6e3 83bc e590 : .....
00035b30: 8de3 81be e381 9fe3 81af e383 91e3 82b9 .....
00035b40: e383 afe3 83bc e383 89e3 818c e981 95e3 .....
```

今回の例ではこんな感じでバイト列がバイナリに含まれていました。

プログラミング言語による痕跡の違い

先ほどのRustの例ではRust関連のエラーメッセージやコンパイラの種類などを確認できました。これらの情報は対象のバイナリがなんの言語で作成されたかを確認するのに有用な方法です。

他の言語での似たような `string` でわかる痕跡の例をあげます。

ソースコードは上のRustコードを該当した言語のコードに変換しました。

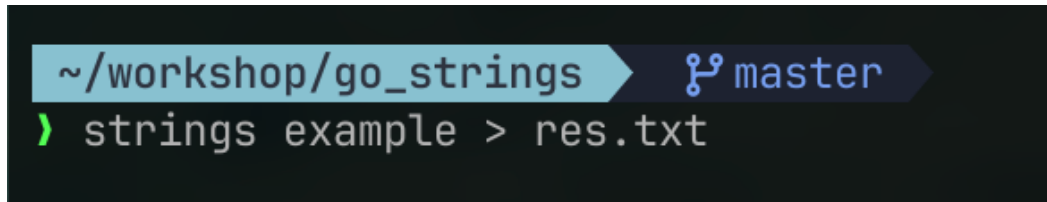
C言語

```
~/workshop/c_strings master
$ strings example
admin
Password_secret
```

情報少なめ

Go言語

Go言語のバイナリは文字列が多すぎるので、次のようにtxtに保存して確認します。



結果のはじめ

```

0+.())M[<1=,{}_Lyi), )(25: .. [("") )
@s -> Pn=[]{}
+TZLLtLuMn"
osnil125625NaNEOPWD../intmapptr...finobigc %: gp *(in n= - P m= MPC=], < end > ]:
???pc= GdupMayUTCaesnettrue3125-Inf+Infopenstatsyncfileboolint8uintchanfuncallkind on in
allgallprootitabsbrkidledead is LEAFbase of ) = <==GOGC] = pc=: p=cas1cas2cas3cas4cas5cas6 at
m= sp= sp: lr: fp= gp= mp=) m=JuneJuly/
etcsha1sha2timefalse<nil>Error1562578125writeclosegetwdlstatint16int32int64uint8arrayslice and
defersweepstRtestWexecWhchanexecRschedsudogtimergscanmheaptracepanicsleeparm64 cnt=gcing MB, got= ...
max=scav ptr ] = (init ms, fault tab= top=[...],
fp.fcnilMarchAprilLocalGreekmullercrc32cpuidStringFormat[]bytesting390625uint16uint32uint64structchan<--
chan Valuesysmontimersefenceselect, not GOROOTobject next= jobs= goid sweep B -> % util alloc free span=
prev= list, i = code= addr=], sp= m->p= p->m=SCHEd curg= ctxt: min= max= bad ts(... )
m=nil base errno SundayMondayFridayAugustCommonsha512cmd/
gonetdnsfloat32float6419531259765625readdirinvaliduintptrChanDir
Value=>forcegcallocmWcpuprofallocmRunknowngctraceIO waitrunningsyscallwaitingforevernetworkUNKNOWN:events,
goid= s=nil
(scan MB in pacer: % CPU ( zombie, j0 = head = panic: nmsys= locks= dying= allocsGODEBUGsignal Signal
m->g0= pad1= pad2= text= minpc=
value= (scan)
types : type TuesdayJanuaryOctoberatomicos/execfips140runtime1s3desGoString48828125nil
PoolscavengepoolDescsyncstttraceBufdeadlockraceFinipanicnilgccheckrunnable is not pointer, errno=
packed=BAD RANK status unknown(trigger= npages= nalloc= nfreed=[signal newval= mcount= bytes, r0 r1
r2 r3 r4 r5 r6 r7 r8 r9 r10 r11 r12 r13 r14 r15
r16 r17 r18 r19 r20 r21 r22 r23 r24 r25 r26 r27 r28 r29
lr sp pc fault
-----
stack=[ minLC= maxpc=
stack=[ minutes status= etypes ThursdaySaturdayFebruaryNovemberDecember%!Month(overflowgo/typesnet/httpgo/
buildneteds0t1smklem method: 244140625complex64interfaceinvalid nreflect: funcargs(bad
indirInterfacewrxrwxrtimerSendpollCacheprofBlockstackpoolhchanLeafwbufSpansGC
(idle)mSpanDeadinittracescavtracepanicwaitchan sendpreemptedcoroutinecopystack -> node= ms cpu, (forced)
wbuf1,n= wbuf2,n= s.limit= s.state= B work ( B exp.) marked unmarked in use)
, size= bad prune, tail= recover: not in [ctxt != 0, oldval=, newval= threads=: status= blocked=
lockedg=atomicor8 runtime= sigcode= m->curg=(unknown)total < 0traceback} stack=[ gp.goid=
lockedm=fdozendirWednesdaySeptemberlocaltimeInheritedmath/randt1rsakex12207031256103515625/dev/
stdincomplex128t.Kind == notifyListprofInsertstackLargeNot workermSpanInUseGOMAXPROCSstop
tracedisablethpinvalidptrschedtracesemacquiredebug call flushGen MB goal, s.state = s.base()=
heapGoal=GOMEMLIMIT KiB now, pages at sweepgen= sweepgen , bound = , limit = Bad varintGC forced
exitThread runqueue= stopwait= runsize= gfreecnt= throwing= spinning=atomicand8float64nanfloat32nan
ntrSize= target= until pc=active < Runknown runtime: goroutine time Date/time LocalWeekday/chart

```

Goのバージョンやどんなコンパイラが使われたかがわかる。

```

go1.24.5
/usr/local/go
call frame too large
expand 32-byte k
MbP?
path
command-line-arguments
build
-buildmode=exe
build
-compiler=gdc
build
CGO_ENABLED=1

```

Swift

```
~/workshop/swift_strings master
> swiftc example.swift -o example

~/workshop/swift_strings master
> strings example
admin
Password_secret
```

Cと同じ情報量だった。

別の章で紹介する `xxd` というコマンドを使えば、`strings` ではできていないCとSwiftの差別化も可能です。

▼ なんて言語によって出力量が違うのか

おそらく**動的リンク**と**静的リンク**が関係していると思われます。

C/Swift=**動的リンク**：OSがライブラリを持ち、実行時に参照する形

Rust/Go=**静的リンク**：実行ファイルにライブラリ・ランタイムを埋め込む形

Rustはどっちもやるみたいです...

ランタイムとはプログラムが動くために必要な基盤のコードのことです。だから Rust/Goはランタイムの出力結果がばり多かった訳です。

stringsはなぜ重要か

マルウェア解析において、安全に実行せずに調べられるかつたくさんの情報を得られることが多いです。

例えば、、、

1：通信先の**C2**サーバーのURL ← 悪い命令を感染したPCに送るまたは盗んだデータを
受け取るサーバーのこと

- 2：ハードコードされた認証情報←AIのおかげで増えそう
- 3：使用ライブラリ・バージョン
- 4：開発環境の情報
- 5：コンパイラ・言語の特定

先ほど私が作成したコードは短かったため、あまり情報は得られませんでした。実際のマルウェアを `strings` するとC言語でもまあまあボリュームです。

変なファイルをダウンロードしてしまった場合は `strings` を行くと、面白い発見があるかもしれません。

まとめ

`strings` とはファイルから人間が読めるASCII文字列を抽出するコマンドで、4文字以上連続した印字可能な文字を先頭から順にスキャンして出力します。

```
strings <ファイル名>
```

`strings` で分かることの例：

ハードコードされた情報やエラーメッセージなどなど

言語によって出力量の違いがある。

C/Swiftだと少なめでRust/Goだと多め

マルウェア解析で実行せずに安全に多くの情報を得ることができるから強い

以下の課題に取り組んでみてください。

問題3：ディレクトリ2で与えられたバイナリファイルの中からあるWebサイトへのURLだと考えられるものを探し、そのURLを教えてください。

問題4：ディレクトリ2で与えられたバイナリファイルがそれぞれなんの言語で書かれたものを特定してみてください。